AI - Assisted Coding
Lab Assignment - 11.1

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

class Stack:

  pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

week 11 > stack.py > ...

```python
#generate a code to demonstrate the stack and its operationns in python
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.stack.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an error if the stack is
        if not self.is_empty():
            return self.stack.pop()
        else:
            raise IndexError("Stack is empty")

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an error if the
        if not self.is_empty():
            return self.stack[-1]
        else:
            raise IndexError("Stack is empty")

    def is_empty(self):
        """Check if the stack is empty."""
        return len(self.stack) == 0

    def size(self):
        """Return the number of items in the stack."""
        return len(self.stack)
```

```
Drive/Desktop/AIAC/week 11/stack.py"
Stack after pushing 1, 2, 3: [1, 2, 3]
Top item (peek): 3
Stack size: 3
Popped item: 3
Stack after popping: [1, 2]
```

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
    pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

#Code :

week 11 > queue.py > ...

```python
#implement data structures queue and its operations in python
class Queue:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.queue.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an error if the queue i
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            raise IndexError("Queue is empty")

    def peek(self):
        """Return the item at the front of the queue without removing it. Raises an error if th
        if not self.is_empty():
            return self.queue[0]
        else:
            raise IndexError("Queue is empty")

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.queue) == 0

    def size(self):
        """Return the number of items in the queue."""
        return len(self.queue)
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/User
Drive/Desktop/AIAC/week 11/queue.py"
Queue after enqueuing 1, 2, 3: [1, 2, 3]
Front item (peek): 1
Queue size: 3
Dequeued item: 1
Queue after dequeuing: [2, 3]
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

```
class Node:
    pass

class LinkedList:
    pass
```

Expected Output:

- A working linked list implementation with clear method documentation.

Code :

week 11 > linkedlist.py > ...

```python
#generate a code to demonstrate the linked list and its operations in python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def display(self):
        elements = []
        current = self.head
        while current:
            elements.append(current.data)
            current = current.next
        return elements
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Us
Drive/Desktop/AIAC/week 11/linkedlist.py"
Linked List elements: [1, 2, 3]
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

Sample Input Code:

class BST:

pass

Expected Output:

- BST implementation with recursive insert and traversal methods.

code:

week 11 > Bst.py > ...

```python
#generate a code to demonstrate the binary search tree and its operations in python
class TreeNode:
    def __init__(self, key):
        self.left = None
        self.right = None
        self.val = key
class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        if self.root is None:
            self.root = TreeNode(key)
        else:
            self._insert_recursively(self.root, key)

    def _insert_recursively(self, node, key):
        if key < node.val:
            if node.left is None:
                node.left = TreeNode(key)
            else:
                self._insert_recursively(node.left, key)
        else:
            if node.right is None:
                node.right = TreeNode(key)
            else:
                self._insert_recursively(node.right, key)

    def inorder_traversal(self):
        return self._inorder_recursively(self.root)
```

Output:

```
Drive/Desktop/AIAC/week 11/Bst.py"
Inorder Traversal of the BST: [2, 3, 4, 5, 6, 7, 8]
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

```
class HashTable:
    pass
```

Expected Output:

- Collision handling using chaining, with well-commented methods.

Code :

week 11 > hashtab.py > ...

```python
#generate a code to demonstrate the hash table and its operations in python
class HashTable:
    def __init__(self):
        self.table = {}

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        self.table[key] = value

    def search(self, key):
        """Search for a value by its key. Returns the value if found, else returns None."""
        return self.table.get(key, None)

    def delete(self, key):
        """Delete a key-value pair from the hash table. Raises an error if the key is not foun
        if key in self.table:
            del self.table[key]
        else:
            raise KeyError("Key not found in hash table")

    def display(self):
        """Display the contents of the hash table."""
        return self.table
```

Output :

```
Drive/Desktop/AIAC/week 11/hashtab.py"
Hash Table after insertions: {'name': 'Alice', 'age': 30, 'city': 'New York'}
Search for 'name': Alice
Search for 'age': 30
Search for 'country' (not found): None
Hash Table after deleting 'age': {'name': 'Alice', 'city': 'New York'}
'Key not found in hash table'
```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

  pass

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

#Code :

week 11 > graph.py > ...

```python
#generate code to demonstrate the graph and its operations in python
class Graph:
    def __init__(self):
        self.graph = {}

    def add_edge(self, u, v):
        """Add an edge between vertices u and v."""
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append(v)
        self.graph[v].append(u)  # For undirected graph

    def display(self):
        """Display the graph as an adjacency list."""
        return self.graph
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Pyt
Drive/Desktop/AIAC/week 11/graph.py"
Graph adjacency list: {1: [2, 3], 2: [1, 4], 3: [1, 4], 4: [2, 3]}
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

  pass

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority) and display methods.

week 11 > priority que.py > ...

```python
#generate code to demonstarte the priority queue and its operations in python
import heapq
class PriorityQueue:
    def __init__(self):
        self._queue = []
        self._index = 0

    def push(self, item, priority):
        heapq.heappush(self._queue, (priority, self._index, item))
        self._index += 1

    def pop(self):
        return heapq.heappop(self._queue)[-1]
    def peek(self):
        return self._queue[0][-1] if self._queue else None
    def is_empty(self):
        return not self._queue
    def size(self):
        return len(self._queue)
```

Output :

```
Drive/Desktop/AIAC/week 11/priority que.py"
Priority Queue after adding tasks: ['task2', 'task1', 'task3']
Peek at highest priority task: task2
Size of the priority queue: 3
Popped item: task2
Priority Queue after popping: ['task3', 'task1']
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

class DequeDS:

  pass

Expected Output:

- Insert and remove from both ends with docstrings.

week 11 > deque.py > ...

```python
#generate a code to demonstrate the deque and its operations in python
class Deque:
    def __init__(self):
        self.deque = []

    def add_front(self, item):
        """Add an item to the front of the deque."""
        self.deque.insert(0, item)

    def add_rear(self, item):
        """Add an item to the rear of the deque."""
        self.deque.append(item)

    def remove_front(self):
        """Remove and return the item at the front of the deque. Raises an error if the deque i
        if not self.is_empty():
            return self.deque.pop(0)
        else:
            raise IndexError("Deque is empty")

    def remove_rear(self):
        """Remove and return the item at the rear of the deque. Raises an error if the deque is
        if not self.is_empty():
            return self.deque.pop()
        else:
            raise IndexError("Deque is empty")

    def peek_front(self):
        """Return the item at the front of the deque without removing it. Raises an error if th
        if not self.is_empty():
            return self.deque[0]
```

#Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppD
Drive/Desktop/AIAC/week 11/deque.py"
Deque after adding 0 at front and 1, 2 at rear: [0, 1, 2]
Front item (peek): 0
Rear item (peek): 2
Deque size: 3
Removed front item: 0
Removed rear item: 2
Deque after removing front and rear items: [1]
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.

entering exiting the campus.

2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification
Student Attendance Tracking	Hash Table	Student attendance can be stored using student ID as a key. Hash tables allow very fast lookup and update of attendance records. This makes it efficient to track daily entry and exit of students.
Event Registration System	Binary Search Tree (BST)	Event participants can be stored in sorted order based on ID or name. BST allows efficient insertion, deletion, and searching operations. This helps manage participants quickly.

Library Book Borrowing	Priority Queue	Some books may have higher priority (e.g., reserved books or urgent academic needs). Priority queue allows books with higher priority to be issued first. This ensures important requests are handled earlier.
Bus Scheduling System	Graph	Bus stops and routes are connected like a network. Graph data structure represents connections between stops effectively. It helps in finding routes between different locations.
Cafeteria Order Queue	Queue	Students are served in the order they arrive at the cafeteria. Queue follows the FIFO (First In First Out) principle. This ensures fair and orderly service.

Student Attendance Tracking :

week 11 > student attendance.py > ...

```python
#generate code for student attendance tracking system in python using hash table
class AttendanceTracker:
    def __init__(self):
        self.attendance = {}

    def mark_attendance(self, student_id, date):
        """Mark attendance for a student on a specific date."""
        if student_id not in self.attendance:
            self.attendance[student_id] = []
        self.attendance[student_id].append(date)

    def get_attendance(self, student_id):
        """Get the attendance record for a specific student."""
        return self.attendance.get(student_id, [])

    def display_attendance(self):
        """Display the attendance records for all students."""
        return self.attendance
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/
Drive/Desktop/AIAC/week 11/student attendance.py"
Attendance for S001: ['2024-06-01', '2024-06-02']
Attendance for S002: ['2024-06-01']
All attendance records: {'S001': ['2024-06-01', '2024-06-02'], 'S002': ['2024-06-01']}
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Library Book Borrowing

week 11 > book borrow.py > Library

```python
#Library Book Borrowing - Keep track of available books and their due dates.
class Library:
    def __init__(self):
        self.books = {}

    def add_book(self, title, due_date):
        self.books[title] = due_date

    def borrow_book(self, title):
        if title in self.books:
            due_date = self.books.pop(title)
            print(f"You have borrowed '{title}'. It is due on {due_date}.")
        else:
            print(f"Sorry, '{title}' is not available.")

    def return_book(self, title, due_date):
        self.books[title] = due_date
        print(f"You have returned '{title}'. It is now available for borrowing.")

    def display_available_books(self):
        if self.books:
            print("Available books:")
            for title, due_date in self.books.items():
                print(f"'{title}' - Due on {due_date}")
        else:
            print("No books are currently available.")
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Pyt
Drive/Desktop/AIAC/week 11/book borrow.py"
You have borrowed 'The Great Gatsby'. It is due on 2024-07-01.
You have borrowed '1984'. It is due on 2024-08-01.
Available books:
'To Kill a Mockingbird' - Due on 2024-07-15
You have returned 'The Great Gatsby'. It is now available for borrowing.
Available books:
'To Kill a Mockingbird' - Due on 2024-07-15
'The Great Gatsby' - Due on 2024-09-01
```

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification
Patient Check-In System	Queue	Patients are treated in the order they arrive. Queue follows the FIFO (First In First Out) principle. This ensures fair processing of patients.
Emergency Case Handling	Priority Queue	Emergency patients must be treated before normal patients. Priority queue processes elements based on priority instead of arrival order. This ensures critical cases are handled first.

Medical Records Storage	Binary Search Tree (BST)	Patient records can be stored based on patient ID. BST allows efficient searching, insertion, and retrieval of records. This helps doctors quickly access patient information.
Doctor Appointment Scheduling	Linked List	Appointments can be added or removed dynamically throughout the day. Linked list allows easy insertion and deletion without shifting data. This makes schedule management flexible.
Hospital Room Navigation	Graph	Hospital rooms and wards are connected like a network. Graph data structure represents connections between rooms effectively. It helps in finding paths between different hospital areas.

week 11 > patient checkin.py > PatientCheckInSystem > queue_size

```python
# generate code for patient Check-In System - Patients are registered and treated in order of
class Patient:
    def __init__(self, name):
        self.name = name
class PatientCheckInSystem:
    def __init__(self):
        self.queue = []

    def check_in(self, patient):
        """Add a patient to the check-in queue."""
        self.queue.append(patient)

    def treat_patient(self):
        """Treat the next patient in the queue. Raises an error if the queue is empty."""
        if not self.is_empty():
            return self.queue.pop(0)
        else:
            raise IndexError("No patients in the queue")

    def peek_next_patient(self):
        """Return the next patient to be treated without removing them from the queue. Raises
        if not self.is_empty():
            return self.queue[0]
        else:
            raise IndexError("No patients in the queue")

    def is_empty(self):
        """Check if there are no patients in the queue."""
        return len(self.queue) == 0

    def queue_size(self):
        """Return the number of patients in the queue."""
```

Output :

```
The Great Gatsby - Due on 2024-09-01
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local
Drive/Desktop/AIAC/week 11/patient checkin.py"
Patients in queue: ['Alice', 'Bob', 'Charlie']
Next patient to be treated (peek): Alice
Number of patients in queue: 3
Treated patient: Alice
Patients in queue after treating one patient: ['Bob', 'Charlie']
```

Doctor Appointment Scheduling

week 11 > docter appointment.py > ...

```python
#generate code for Doctor Appointment Scheduling - Appointments sorted by time use priority
import heapq
class DoctorAppointment:
    def __init__(self):
        self.appointments = []
        self._index = 0

    def schedule_appointment(self, patient_name, appointment_time):
        """Schedule an appointment for a patient at a specific time."""
        heapq.heappush(self.appointments, (appointment_time, self._index, patient_name))
        self._index += 1

    def next_appointment(self):
        """Return the next appointment without removing it from the schedule."""
        return self.appointments[0] if self.appointments else None

    def attend_appointment(self):
        """Attend the next appointment and remove it from the schedule."""
        return heapq.heappop(self.appointments) if self.appointments else None

    def is_schedule_empty(self):
        """Check if there are no appointments scheduled."""
        return not self.appointments

    def schedule_size(self):
        """Return the number of appointments scheduled."""
        return len(self.appointments)
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/Us
Drive/Desktop/AIAC/week 11/docter appointment.py"
Appointments scheduled: [('2024-07-01 09:00', 'Bob'), ('2024-07-01 10:00', 'Alice'), ('2024-07-01 11:00', 'Charlie')]
Next appointment (peek): ('2024-07-01 09:00', 1, 'Bob')
Number of appointments scheduled: 3
Attended appointment: ('2024-07-01 09:00', 1, 'Bob')
Appointments after attending one: [('2024-07-01 10:00', 'Alice'), ('2024-07-01 11:00', 'Charlie')]
Is the schedule empty? False
Is the schedule empty after attending all appointments? True
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification
Traffic Signal Queue	Queue	Vehicles wait at traffic signals in the order they arrive. Queue follows the FIFO (First In First Out) principle. This ensures vehicles move in an organized order.

Emergency Vehicle Priority Handling	Priority Queue	Ambulances and fire trucks must move before normal vehicles. Priority queue allows vehicles with higher priority to be processed first. This helps emergency vehicles pass quickly.
Vehicle Registration Lookup	Hash Table	Vehicle details can be stored using the registration number as a key. Hash tables allow very fast search and retrieval of records. This makes vehicle lookup efficient.
Road Network Mapping	Graph	Roads connect intersections forming a network. Graph data structure represents connections between different roads and junctions. It helps in route planning and navigation.
Parking Slot Availability	Stack	In some parking systems, the last parked vehicle exits first. Stack follows the LIFO (Last In First Out) principle. This makes it suitable for parking management in limited spaces.

Traffic Signal Queue:

week 11 > traffic signal.py > ...

```python
#generate code for traffic signal queue using queue data structure in python
from collections import deque
class TrafficSignalQueue:
    def __init__(self):
        self.queue = deque()

    def add_vehicle(self, vehicle):
        """Add a vehicle to the traffic signal queue."""
        self.queue.append(vehicle)

    def remove_vehicle(self):
        """Remove and return the vehicle at the front of the queue. Raises an error if the queu
        if not self.is_empty():
            return self.queue.popleft()
        else:
            raise IndexError("Traffic signal queue is empty")

    def peek_vehicle(self):
        """Return the vehicle at the front of the queue without removing it. Raises an error if
        if not self.is_empty():
            return self.queue[0]
        else:
            raise IndexError("Traffic signal queue is empty")

    def is_empty(self):
        """Check if the traffic signal queue is empty."""
        return len(self.queue) == 0

    def size(self):
        """Return the number of vehicles in the traffic signal queue."""
        return len(self.queue)
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Pyth
Drive/Desktop/AIAC/week 11/traffic signal.py"
Traffic signal queue after adding vehicles: ['Car', 'Bus', 'Motorcycle']
Next vehicle to pass (peek): Car
Number of vehicles in queue: 3
Vehicle passing through the signal: Car
Traffic signal queue after one vehicle passes: ['Bus', 'Motorcycle']
Is the traffic signal queue empty? False
Is the traffic signal queue empty after all vehicles pass? True
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Parking Slot Availability :

week 11 > parking slot.py > ParkingLot > park_car

```python
#generate code for Parking Slot Availability using stack
class ParkingLot:
    def __init__(self, capacity):
        self.capacity = capacity
        self.stack = []

    def park_car(self, car_number):
        if len(self.stack) < self.capacity:
            self.stack.append(car_number)
            print(f"Car {car_number} parked.")
        else:
            print("Parking lot is full. Cannot park the car.")

    def leave_parking(self):
        if self.stack:
            car_number = self.stack.pop()
            print(f"Car {car_number} left the parking lot.")
        else:
            print("Parking lot is empty. No cars to leave.")

    def available_slots(self):
        return self.capacity - len(self.stack)
```

```
Drive/Desktop/AIAC/week 11/parking slot.py"
Car ABC123 parked.
Car XYZ789 parked.
Car LMN456 parked.
Available slots: 2
Car LMN456 left the parking lot.
```

:

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Feature	Data Structure	Justification
Shopping Cart Management	Linked List	Products can be added or removed dynamically from the shopping cart. Linked lists allow easy insertion and deletion without shifting elements. This makes cart management flexible.
Order Processing System	Queue	Orders must be processed in the order they are placed. Queue follows the FIFO (First In First Out) principle. This ensures fair order processing.
Top-Selling Products Tracker	Priority Queue	Products can be ranked based on the number of sales. Priority queue helps retrieve the highest selling products quickly. This is useful for displaying trending products.
Product Search Engine	Hash Table	Products can be searched using product ID. Hash tables provide very fast lookup operations. This allows quick retrieval of product details.
Delivery Route Planning	Graph	Warehouses and delivery locations are connected like a network. Graph data structure represents these connections effectively. It helps find the best delivery routes.

Shopping Cart Management :

week 11 > shopping cart.py > ...

```python
#generate a code for Shopping Cart Management using linkedin list
class Node:
    def __init__(self, item):
        self.item = item
        self.next = None
class ShoppingCart:
    def __init__(self):
        self.head = None

    def add_item(self, item):
        new_node = Node(item)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node

    def remove_item(self, item):
        current = self.head
        previous = None
        while current:
            if current.item == item:
                if previous:
                    previous.next = current.next
                else:
                    self.head = current.next
                return True  # Item removed
            previous = current
            current = current.next
        return False  # Item not found
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Pyth
Drive/Desktop/AIAC/week 11/shopping cart.py"
Shopping Cart Items: ['Apple', 'Banana', 'Orange']
Shopping Cart Items after removing Banana: ['Apple', 'Orange']
Shopping Cart Items after attempting to remove Grapes: ['Apple', 'Orange']
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Top-Selling Products Tracker:

week 13 > top selling.py > ...

```python
#generate a code for Top-Selling Products Tracker using priority queue
import heapq
class TopSellingProductsTracker:
    """A class to track the top-selling products using a priority queue."""

    def __init__(self):
        """Initializes the TopSellingProductsTracker with an empty priority queue."""
        self.products = []

    def add_product(self, product_name, sales):
        """
        Adds a product and its sales to the tracker.

        Args:
            product_name (str): The name of the product.
            sales (int): The number of sales for the product.
        """
        heapq.heappush(self.products, (-sales, product_name))

    def get_top_selling_products(self, n):
        """
        Retrieves the top N selling products.

        Args:
            n (int): The number of top-selling products to retrieve.

        Returns:
            list: A list of tuples containing the product name and sales.
        """
        top_products = heapq.nsmallest(n, self.products)
        return [(product_name, -sales) for sales, product_name in top_products]
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/User
Drive/Desktop/AIAC/week 13/top selling.py"
[('Product B', 150), ('Product C', 120)]
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification
Shipment Processing	Queue	Shipments are processed in the order they arrive. Queue follows the FIFO (First In First Out) principle. This ensures fair and systematic shipment handling.

Urgent Shipment Priority	Priority Queue	Some shipments may be urgent and need faster delivery. Priority Queue processes shipments based on priority instead of arrival order. This ensures urgent shipments are delivered first.
Product Lookup by Code	Binary Search Tree (BST)	Products can be stored using their product code in a sorted structure. BST allows efficient searching, insertion, and deletion operations. This helps quickly retrieve product information.
Warehouse Network Connectivity	Graph	Warehouses and transport routes are connected like a network. Graph data structure represents connections between warehouses efficiently. It helps find routes between different warehouses.
Inventory Add/Remove Operations	Linked List	Inventory items are frequently added or removed. Linked List allows easy insertion and deletion without shifting elements. This makes inventory management flexible.

Warehouse Network Connectivity :

week 13 > warehouse connect.py > WarehouseNetwork > are_connected > visited

```python
# write a python program for Warehouse Network Connectivity using graphs
class WarehouseNetwork:
    """A class to represent a warehouse network using a graph."""

    def __init__(self):
        """Initializes the WarehouseNetwork with an empty graph."""
        self.graph = {}

    def add_warehouse(self, warehouse_name):
        """
        Adds a warehouse to the network.

        Args:
            warehouse_name (str): The name of the warehouse.
        """
        if warehouse_name not in self.graph:
            self.graph[warehouse_name] = []

    def connect_warehouses(self, warehouse1, warehouse2):
        """
        Connects two warehouses in the network.

        Args:
            warehouse1 (str): The name of the first warehouse.
            warehouse2 (str): The name of the second warehouse.
        """
        if warehouse1 in self.graph and warehouse2 in self.graph:
            self.graph[warehouse1].append(warehouse2)
            self.graph[warehouse2].append(warehouse1)

```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/Use
Drive/Desktop/AIAC/week 13/warehouse connect.py"
True
False
```

Inventory Add/Remove Operations:

week 13 > inventor.py > ...

```python
#generate a code for Inventory Add/Remove Operations using  linked list
class InventoryNode:
    """A node in the inventory linked list."""

    def __init__(self, product_name, quantity):
        """
        Initializes the InventoryNode with the given product name and quantity.

        Args:
            product_name (str): The name of the product.
            quantity (int): The quantity of the product in stock.
        """
        self.product_name = product_name
        self.quantity = quantity
        self.next = None
class InventoryLinkedList:
    """A linked list to manage inventory add/remove operations."""

    def __init__(self):
        """Initializes the InventoryLinkedList with an empty head."""
        self.head = None

    def add_product(self, product_name, quantity):
        """
        Adds a product to the inventory. If the product already exists, it updates the quantity

        Args:
            product_name (str): The name of the product.
            quantity (int): The quantity of the product to add.
        """
        if self.head is None:
            self.head = InventoryNode(product_name, quantity)
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C
Drive/Desktop/AIAC/week 13/inventor.py"
Current Inventory:
Product: Product A, Quantity: 150
Product: Product B, Quantity: 150

Inventory after removing Product A:
Product: Product A, Quantity: 30
Product: Product B, Quantity: 150

Inventory after removing Product B:
Product: Product A, Quantity: 30
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

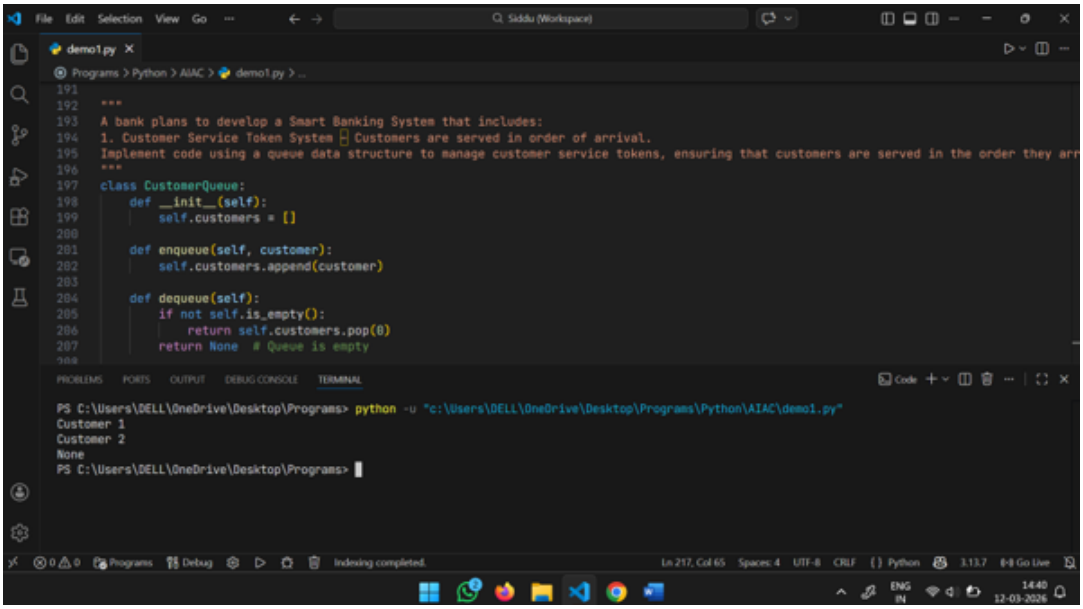
1. **Customer Service Token System** – Customers are served in order of arrival.
2. **Loan Approval Priority Handling** – High-value or urgent loans processed first.
3. **Account Information Lookup** – Fast retrieval of customer details using Account Number.
4. **Transaction History Management** – Maintain chronological transaction records.
5. **ATM Network Connectivity** – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Feature	Data Structure	Justification

Customer Service Token System	Queue	Customers are served in the order they arrive at the bank. Queue follows the FIFO (First In First Out) principle. This ensures fair service and organized customer handling.
Loan Approval Priority Handling	Priority Queue	Some loan requests may be urgent or high value. Priority Queue processes requests based on priority instead of arrival order. This allows important loans to be processed first.
Account Information Lookup	Hash Table	Customer account details can be stored using account numbers as keys. Hash tables allow very fast searching and retrieval of information. This helps bank staff quickly access customer records.
Transaction History Management	Binary Search Tree (BST)	Transactions can be stored in sorted order using transaction ID or time. BST allows efficient insertion and searching operations. This helps manage and retrieve transaction records easily.
ATM Network Connectivity	Graph	ATMs in different cities are connected like a network. Graph data structure represents connections between ATMs efficiently. It helps in managing routes and connectivity between locations.



Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. **Student Login Authentication** – Quick validation of user credentials.
2. **Assignment Submission Queue** – Submissions processed in order of upload.
3. **Top Performer Ranking** – Rank students based on scores.
4. **Course Prerequisite Structure** – Represent subject dependencies.
5. **Undo Feature in Online Editor** – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

Feature	Data Structure	Justification
Student Login Authentication	Hash Table	Student login credentials can be stored using username as a key. Hash tables allow very fast lookup and validation of user information. This makes authentication quick and efficient.
Assignment Submission Queue	Queue	Assignments are processed in the order they are submitted by students. Queue follows the FIFO (First In First Out) principle. This ensures fair processing of submissions.

Top Performer Ranking	Priority Queue	Students can be ranked based on their scores. Priority Queue helps retrieve students with highest marks first. This is useful for displaying top performers.
Course Prerequisite Structure	Graph	Courses may depend on other prerequisite courses. Graph structure represents relationships between subjects. It helps track course dependencies easily.
Undo Feature in Online Editor	Stack	Stack follows the LIFO (Last In First Out) principle. The last action performed can be undone first. This makes it suitable for implementing undo functionality.

week 13 > demo1.py > ...

```python
class AssignmentQueue:
    """Queue implementation for handling assignment submissions.

    This class manages assignment submissions from students.
    Submissions are processed in the order they are received
    using the FIFO (First In First Out) principle.
    """

    def __init__(self):
        """Initialize an empty assignment queue."""
        self.queue = []

    def submit_assignment(self, student):
        """Add a student's assignment to the queue.

        Args:
            student (str): Name of the student submitting the assignment.
        """
        self.queue.append(student)
        print(student, "submitted assignment")

    def process_submission(self):
        """Process the first assignment in the queue.

        Removes the earliest submitted assignment and marks it as checked.
        """
        if self.queue:
            student = self.queue.pop(0)
            print(student, "assignment checked")
        else:
            print("No submissions available")
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/Us
Drive/Desktop/AIAC/week 13/demo1.py"
Rahul submitted assignment
Anita submitted assignment
Vikram submitted assignment
Submission Queue: ['Rahul', 'Anita', 'Vikram']
Rahul assignment checked
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task Description #16: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. **Passenger Check-In Queue** – Process passengers in order of arrival.
2. **Emergency Landing Priority** – Aircraft in emergency get priority.
3. **Flight Information Lookup** – Fast search using Flight ID.
4. **Air Route Connectivity** – Airports connected via routes.
5. **Boarding Process (Last-In First-Out Cabin Storage)** – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings

Feature	Data Structure	Justification
Passenger Check-In Queue	Queue	Passengers are processed in the order they arrive at the airport check-in counter. Queue follows the FIFO (First In First Out) principle. This ensures fair and orderly passenger processing.
Emergency Landing Priority	Priority Queue	Aircraft in emergency situations must land before normal flights. Priority Queue allows aircraft with higher priority to be processed first. This ensures safety and quick response.

Flight Information Lookup	Hash Table	Flight details can be stored using flight ID as a key. Hash tables allow very fast lookup operations. This helps airport staff quickly retrieve flight information.
Air Route Connectivity	Graph	Airports are connected through flight routes forming a network. Graph data structure represents these connections effectively. It helps in managing and analyzing flight paths.
Boarding Process (Cabin Storage Simulation)	Stack	Overhead cabin storage often follows a last-in first-out pattern. Stack allows the most recently placed luggage to be removed first. This simulates luggage handling in overhead bins.

week 13 > demo1.py > ...

```python
class PassengerQueue:
    """Queue implementation for airport passenger check-in system.

    This class manages passengers arriving at the airport.
    Passengers are processed in the order they arrive
    following the FIFO (First In First Out) principle.
    """

    def __init__(self):
        """Initialize an empty passenger queue."""
        self.queue = []

    def add_passenger(self, name):
        """Add a passenger to the check-in queue.

        Args:
            name (str): Name of the passenger.
        """
        self.queue.append(name)
        print(name, "added to check-in queue")

    def process_passenger(self):
        """Process the first passenger in the queue."""
        if self.queue:
            passenger = self.queue.pop(0)
            print(passenger, "checked in successfully")
        else:
            print("No passengers in queue")

    def display(self):
        """Display all passengers waiting in queue."""
        print("Passenger Queue:", self.queue)
```

Output :

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Use
Drive/Desktop/AIAC/week 13/demo1.py"
Rahul added to check-in queue
Anita added to check-in queue
Vikram added to check-in queue
Passenger Queue: ['Rahul', 'Anita', 'Vikram']
Rahul checked in successfully
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task Description #17: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

1. **News Feed Generation** – Display posts chronologically.
2. **Trending Hashtags Tracker** – Rank hashtags by usage frequency.
3. **User Profile Search** – Quick lookup using username.
4. **Friend Network Representation** – Represent user connections.
5. **Message Undo Feature** – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments.

Feature	Data Structure	Justification
News Feed Generation	Linked List	Posts appear chronologically and new posts are continuously added. Linked lists allow easy insertion of new posts without shifting elements. This helps maintain the feed efficiently.
Trending Hashtags Tracker	Priority Queue	Hashtags can be ranked based on frequency of usage. Priority queue allows the most frequently used hashtags to appear first. This helps display trending topics quickly.
User Profile Search	Hash Table	User profiles can be stored using usernames as keys. Hash tables allow very fast lookup and retrieval. This helps quickly find user profiles.

Friend Network Representation	Graph	Users and their friendships form a network structure. Graph represents connections between users effectively. It helps analyze relationships and connections between people.
Message Undo Feature	Stack	Stack follows the LIFO (Last In First Out) principle. The most recent message action can be undone first. This makes it suitable for undo functionality.

week 13 > demo1.py > ...

```python
class MessageStack:
    """Stack implementation for message undo feature.

    This class stores sent messages and allows
    the most recent message to be undone using
    the LIFO (Last In First Out) principle.
    """

    def __init__(self):
        """Initialize an empty message stack."""
        self.stack = []

    def send_message(self, message):
        """Add a message to the stack.

        Args:
            message (str): The message sent by the user.
        """
        self.stack.append(message)
        print("Message sent:", message)

    def undo_message(self):
        """Undo the most recently sent message."""
        if self.stack:
            message = self.stack.pop()
            print("Message undone:", message)
        else:
            print("No messages to undo")

    def display(self):
        """Display all messages in the stack."""
        print("Messages:", self.stack)
```

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/Use
Drive/Desktop/AIAC/week 13/demo1.py"
Message sent: Hello
Message sent: How are you?
Message sent: See you soon
Messages: ['Hello', 'How are you?', 'See you soon']
Message undone: See you soon
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification

Room Allotment	Queue	Students are allotted hostel rooms in the order they apply. Queue follows the FIFO (First In First Out) principle. This ensures fair and organized room allocation.
Medical Emergency Handling	Priority Queue	Students with medical emergencies must receive immediate attention. Priority Queue processes cases based on urgency instead of arrival order. This ensures critical situations are handled first.
Student Record Retrieval	Hash Table	Student records can be stored using student ID as a key. Hash tables allow very fast lookup and retrieval operations. This helps administrators access records quickly.
Hostel Block Connectivity	Graph	Hostel blocks and facilities are connected like a network. Graph data structure represents these connections effectively. It helps in navigation and infrastructure planning.
Visitor Entry Log	Stack	The most recent visitor entry may need to be checked or removed first. Stack follows the LIFO (Last In First Out) principle. This makes it useful for maintaining visitor logs.

week 13 > demo1.py > ...

```python
class HostelQueue:
    """Queue implementation for hostel room allotment.

    This class manages students waiting for hostel rooms.
    Rooms are allotted in the order students apply
    following the FIFO (First In First Out) principle.
    """

    def __init__(self):
        """Initialize an empty hostel queue."""
        self.queue = []

    def add_student(self, name):
        """Add a student to the hostel waiting list.

        Args:
            name (str): Name of the student.
        """
        self.queue.append(name)
        print(name, "added to hostel waiting list")
```

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User
Drive/Desktop/AIAC/week 13/demo1.py"
Rahul added to hostel waiting list
Anita added to hostel waiting list
Vikram added to hostel waiting list
Hostel Waiting List: ['Rahul', 'Anita', 'Vikram']
Room allotted to Rahul
PS C:\Users\User\OneDrive\Desktop\AIAC>
```

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

- For each feature, select the most appropriate data structure from the list below:
  - Stack
  - Queue
  - Priority Queue
  - Linked List
  - Binary Search Tree (BST)
  - Graph
  - Hash Table
  - Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Feature	Data Structure	Justification
Complaint Handling	Queue	Consumer complaints are handled in the order they are received. Queue follows the FIFO (First In First Out) principle. This ensures fair processing of complaints.

Fault Priority Handling	Priority Queue	Some faults may be critical and require immediate attention. Priority Queue processes issues based on urgency rather than arrival order. This ensures serious problems are resolved first.
Consumer Record Lookup	Hash Table	Consumer information can be stored using consumer ID as a key. Hash tables allow very fast lookup and retrieval. This helps quickly access consumer details.
Power Grid Connectivity	Graph	Power stations and substations are connected like a network. Graph data structure represents these connections effectively. It helps analyze and manage power distribution routes.
Meter Reading History Tracking	Linked List	Meter readings are recorded regularly over time. Linked lists allow easy insertion of new readings without shifting data. This helps maintain a chronological record.

week 13 > demo1.py > ...

```python
class ComplaintQueue:
    """Queue implementation for electricity complaint handling.

    This class manages consumer complaints in the order
    they are received using the FIFO (First In First Out) principle.
    """

    def __init__(self):
        """Initialize an empty complaint queue."""
        self.queue = []

    def register_complaint(self, complaint):
        """Add a complaint to the queue.

        Args:
            complaint (str): Complaint description from consumer.
        """
        self.queue.append(complaint)
        print("Complaint registered:", complaint)
```

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Python/pythoncore-3.14-64/pyth
Drive/Desktop/AIAC/week 13/demo1.py"
Complaint registered: Power outage in Block A
Complaint registered: Low voltage in Street 5
Complaint registered: Meter not working
Pending Complaints: ['Power outage in Block A', 'Low voltage in Street 5', 'Meter not working']
Complaint resolved: Power outage in Block A
PS C:\Users\User\OneDrive\Desktop\AIAC>
```